Predicting Coronary Heart Disease from Lifestyle Survey Data with Weighted Logistic Regression

Yvan Bénon, Grégoire Taillard, Jean Erard
CS-433 Machine Learning, EPFL, Switzerland

***Abstract*—We predict whether a person has been diagnosed with coronary heart disease (MICHD) from 321 answers to the BRFSS health survey. The data is noisy (many missing values, coded categorical answers) and strongly imbalanced (8.8% of positive cases), so that the six basic methods seen in class reach an F1 score below 0.10. With a preprocessing pipeline learned on the training data only, an L2-regularized logistic regression trained with Newton's method, and a decision threshold tuned by cross-validation, we reach an F1 score of 0.433 on a hold-out set and 0.438 (accuracy 0.869) on the AIcrowd test set. An ablation study shows that handling the class imbalance is by far the most important ingredient.**

I. Introduction

Cardiovascular diseases are a leading cause of death worldwide. The Behavioral Risk Factor Surveillance System (BRFSS) is a telephone survey of U.S. residents about their health and lifestyle. Given the answers of a person, our task is a binary classification: predict $y = 1$ if the person has MICHD (heart attack or angina) and $y = -1$ otherwise. The training set contains $N = 328{,}135$ people and $D = 321$ features, the test set 109,379 people. Since only 8.8% of the labels are positive, a model predicting always -1 has 91% accuracy but is useless; we therefore optimize and report the F1 score of the positive class, the metric of the competition. Only NumPy is used.

II. Models and Methods

A. Evaluation protocol

The training set is randomly split (seed 42) into a 60% *training part* and a 40% *hold-out part*. All preprocessing statistics and hyperparameters are chosen on the training part only (hyperparameters by 5-fold cross-validation), and all the numbers of Tables I and II are measured on the hold-out part. The final model is retrained on the full training set with the selected hyperparameters (`run.py`).

B. Preprocessing

Many features have a large fraction of missing values (68 features have at least 95% of NaN) and many are categorical answers coded as numbers. The pipeline (`preprocessing.py`) is fitted on the training data and applied unchanged to the other sets:

1) drop the features with $\geq 95\%$ missing values;
2) drop the almost constant features (most frequent value in $\geq 99\%$ of the non-missing entries);
3) *one-hot encode* the non-binary features with at most 5 distinct values (missing counted as a value): one indicator per category except a reference one, plus a missing-value indicator if needed;
4) standardize the other features and replace their missing values by 0, i.e. by the mean;
5) drop the features whose absolute correlation with a previous feature is ≥ 0.99, and add an intercept.

This turns the 321 raw features into $D' = 417$ features.

C. Baselines

We first train the six methods of `implementations.py`: linear regression by gradient descent (GD, 100 steps, $\gamma = 0.02$), by stochastic gradient descent (SGD, 10^4 steps, $\gamma = 10^{-3}$), by the normal equations (least squares) and with an L2 penalty (ridge, $\lambda = 2.2 \cdot 10^{-3}$ chosen by 5-fold CV on the RMSE), and logistic regression without and with penalty ($\lambda = 10^{-4}$) by gradient descent (100 steps, $\gamma = 0.1$, labels mapped to $\{0, 1\}$). The step sizes are below the stability limits $2/L$ and $8/L$, where $L \approx 51$ is the largest eigenvalue of $X^\top X/N$. All predict 1 if $\mathbf{x}^\top \mathbf{w} \geq 0$.

D. Our model

We minimize a class-weighted, L2-regularized logistic loss

$$\mathcal{L}(\mathbf{w}) = \frac{1}{N} \sum_{n=1}^{N} c_n \log\Big(1 + e^{-y_n \mathbf{x}_n^\top \mathbf{w}}\Big) + \lambda \|\mathbf{w}\|^2, \qquad (1)$$

with balanced weights $c_n = N/(2N_{y_n})$, where N_{y_n} is the number of samples of the class of n, so that both classes have the same total weight. Since D' is small, we use *Newton's method*: each step solves $H\,\Delta\mathbf{w} = \nabla\mathcal{L}$ with $H = \frac{1}{N} X^\top S X + 2\lambda I$, $S = \mathrm{diag}(c_n \sigma_n (1 - \sigma_n))$, $\sigma_n = \sigma(\mathbf{x}_n^\top \mathbf{w})$. It converges in about 10 steps and needs no step size. A sample is classified as positive if $\mathbf{x}^\top \mathbf{w} \geq t$.

The two hyperparameters are chosen by 5-fold cross-validation on the training part: $\lambda \in \{10^{-8}, \ldots, 10^{-3}\}$ (10 values, log scale) minimizing the validation logistic loss, giving $\lambda = 2.15 \cdot 10^{-5}$; then $t \in \{-2, -1.9, \ldots, 1.9\}$ maximizing the validation F1 score, giving $t = 0.9$ (Fig. 1). The final model uses 15 Newton steps.

Table I
THE SIX BASIC METHODS ON THE HOLD-OUT SET (THRESHOLD 0).

Method	F1	Accuracy
Least squares GD	0.001	0.911
Least squares SGD	0.017	0.911
Least squares (normal equations)	0.092	0.913
Ridge regression	0.079	0.913
Logistic regression (GD)	0.096	0.912
Reg. logistic regression (GD)	0.096	0.912

Table II
ABLATION STUDY ON THE HOLD-OUT SET: LOGISTIC REGRESSION TRAINED BY NEWTON'S METHOD ($\lambda = 2.15 \cdot 10^{-5}$). THRESHOLD t TUNED BY 5-FOLD CV WHEN MARKED "TUNED". LAST COLUMN: FRACTION OF PREDICTED POSITIVES.

Variant	t	F1	Acc.	Pos.
No weights, $t = 0$	0	0.242	0.914	2.4%
Class weights, $t = 0$	0	0.379	0.765	28.9%
No weights, tuned t	-1.4	0.432	0.872	13.6%
Class weights, tuned t (final)	0.9	**0.433**	0.866	14.8%
without one-hot encoding	0.9	0.417	0.858	15.3%

III. RESULTS

Table I shows that all basic methods reach an accuracy of about 91%, i.e. the accuracy of always predicting -1: because of the imbalance, almost all scores are negative and the F1 score stays below 0.10. The GD and SGD versions of least squares are even worse than the closed-form solution, as they are far from convergence with a stable step size.

Table II isolates the effect of each modification. Simply training the logistic model to convergence with Newton's method already improves the F1 score to 0.242, but it predicts only 2.4% of positives. Balanced class weights raise it to 0.379 by shifting the scores, but now predict too many positives (28.9%). Tuning the threshold fixes the proportion of positives to about 14%, which maximizes the F1 score (Fig. 1), and gives 0.433. One-hot encoding the categorical features adds $+0.016$ in F1 compared to treating their codes as numbers. On the AIcrowd test set, the final model (retrained on all the data) obtains **F1 = 0.438** and accuracy 0.869, consistent with the hold-out estimate (0.433 and 0.866), so the model does not overfit.

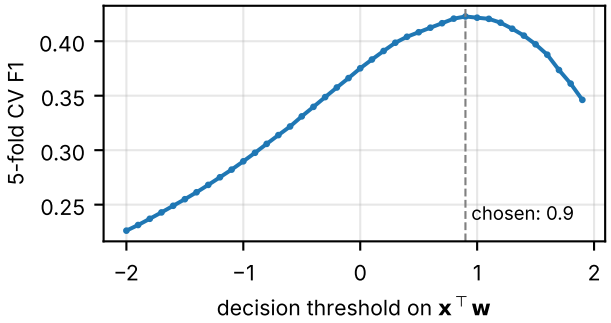


Figure 1. Mean 5-fold cross-validation F1 score of the final model as a function of the decision threshold t.

IV. DISCUSSION

The main difficulty of this dataset is the class imbalance, not the model complexity: a linear model reaches an F1 score of 0.43 once its decision rule is adapted, whereas a better fit of the loss alone does not help. Interestingly, class weights and threshold tuning are almost interchangeable: with a tuned threshold, the unweighted model is only 0.001 below the weighted one. Both mainly move the decision boundary; tuning the threshold directly on the F1 score is the simplest and most important step.

The model has limits. The threshold optimizes F1, which favors recall over accuracy; a medical application could prefer another trade-off. The preprocessing is generic: the special survey codes (e.g. 7/77 "don't know", 9/99 "refused", 88 "none") are kept as numbers, and categorical features with more than 5 values (e.g. the state) are treated as continuous. Finally, the preprocessing is fitted once on the training part before cross-validation, which slightly leaks information into the validation folds; the agreement between hold-out and test scores suggests that this effect is small.

V. SUMMARY

A careful preprocessing, a weighted L2-regularized logistic regression trained with Newton's method and a decision threshold tuned by cross-validation raise the F1 score from below 0.10 for the basic methods to 0.433 on a hold-out set and 0.438 on the test set. Handling the class imbalance accounts for most of the gain, and one-hot encoding of the categorical answers for a smaller part. All results are reproduced by `run.py`.